

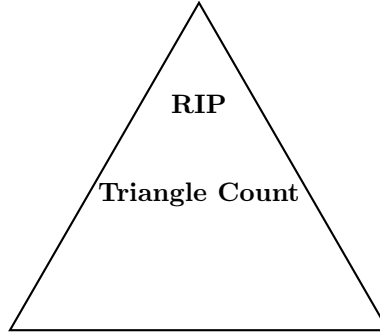
HOPE: The Greatest of All Treasures

Ullas ☺

*Department of Computer Science and Automation,
Indian Institute of Science, Bangalore
email: ullas@iisc.ac.in*

Abstract

Triangle Counting is a seemingly important problem in various domains, which the authors haven't bothered to study. Nevertheless, we know it is. Trust us. In this paper, we present a family of novel algorithms to solve this problem in a parallel setting on the GPU. We show, er..., wherever we can, the performance improvement that we reap from this family of algorithms. Kindly note that we present not just one algorithm, but a *family* of them, which alone should bestow the most coveted Best Paper Award, for this paper is worth many. We show that we can achieve time complexity of $\mathcal{O}(1)$, beating every known competing algorithm in the literature, past, present and future, in every alternate universe you can conceive of. We choose to call this family of algorithms HOPE.



The above tombstone lays to rest the entire field of Triangle Counting, owing to the dazzling brilliance of this paper. For any curious reader who's wondering why the above tombstone is not shaped like a tombstone, please remember that it's a tombstone for *Triangle* Counting, and hence, due respects must be paid.

1 Introduction

Given an undirected graph $G = (V, E)$ where V represents the set of Vertices and E the set of Edges, we would like to count the number of triangles present in the graph. Actually,

we wouldn't like to, given the choice, but the problem exists. For the totally uninitiated, an undirected graph is one which doesn't have very focussed goals in life. A trivial approach would be to traverse the graph, visiting each triple of nodes and checking if there are edges between them all, incrementing a counter if so. The value of the counter at the end would be the Triangle Count. This algorithm is prohibitively expensive, however, owing to $\mathcal{O}(n^3)$ time complexity. We would like to tackle this in a parallel setting (on a GPU), so as to reduce the time complexity. We present in this work a family of algorithms (which to be honest, aren't really related to each other except by the title we chose to give it) to solve this problem. We call this family HOPE (because after all, it is a well known fact [1] that Hope is the greatest of all treasures).

2 Related Work

Who cares?

3 Algorithm-1: Highly Optimized Parallel Encoding

Assign each thread to a triple of vertices. If there are n vertices in the graph, there are n^3 triples, and so, spawn n^3 threads. Each thread then checks if the triple is connected by edges to each other, and if so, a global counter is incremented.

Reviewer-2: The graph can be really huge. n can be a billion. In which case, n^3 would be 10^{27} .

Author: We agree.

Reviewer-2: How would you spawn so many threads?

Author: Give them a long rope to hang themselves with.

Reviewer-2: That doesn't answer the question.

We acknowledge the valid concern raised, and hence we DO NOT suggest, as any naive reader (or reviewer) would, that we can partition the set of all triples into multiple chunks, and at a time assign as many chunks to threads as the system capacity allows to spawn threads (handling partitions in sequence), keeping track of which triple has been already checked by marking it off a bitvector, etc. etc. This is the easy and naive way. It also works. Where is the fun in that? Besides, it's a classical approach, so we don't recall it. No, we don't mean to say that we don't repeat it here in the interests of space, but that we actually don't *recall* it.

No, what we DO suggest is the following optimization: encode each triple to a number. This number indicates whether or not the triple forms a triangle in the graph. This can be done in parallel, clearly. Now, all we need to do is count how many such triples are there, inspecting their encoding. To tackle the challenge of spawning n^3 threads, we only spawn as many threads as we can, and let each thread inspect the encoding, incrementing the global counter if a triangle is found, by a number k . This number k indicates how many other triples are part of triangles. Because, oh, we forgot to mention, that is what each encoding of a triple is all about! The encoding is an acknowledgement of limits of thread-spawning capacity of the system, thereby allowing each triple to also keep track of other triples' encoding.

Reviewer-3: How can you come up with such a clever encoding?

Author: Thank you for the compliment!

Reviewer-3: That wasn't a rhetoric, I was actually asking a question.

Author: Oh! We simply HOPE that such an encoding exists.

4 Algorithm-2: Have Overapproximate Problem Enhancement

What's the harm in finding more than just triangles? Why would you be so besotted about triangles, huh? If we can find bigger polygons, wouldn't that be better? And so, let us enhance the problem into finding the largest clique in the graph. We know that this is a harder problem than simply triangle count. We also know that if we can solve this much harder problem, triangle count would be a piece of cake. Oh, but we haven't started out by taking on the task of solving the harder problem of largest clique. Therefore, this simply allows us to refer [2], which is a reference that we HOPE will solve the problem someday in the future.

5 Algorithm-3: Have Optimistic Processing Engines

<Begin Digression>

The idea is simple. It is that the Worst Case Time Complexity is a misnomer. To understand why this is so, consider the simple example of linear search in an array (in a single-threaded environment). What is the time complexity of Linear Search? $\mathcal{O}(n)$, you would answer, in the blink of an eye. Ah, but you see, in the blink of an eye, things can change! What if, while you are searching for a non-existent element x in the array, after you have iterated over some element i , the i^{th} element suddenly becomes x , owing to capacitor discharge, or leaky transistors, or quantum fluctuations, or any of the gamut of glitches that can plague as complex a system as a computer? You would have wrongly answered that the element is not present when in fact, it has suddenly become present.

Reviewer-1: We will assume that such things don't happen.

Author: What is the Worst Case Time Complexity then?

Reviewer-1: $\mathcal{O}(n)$.

WRONG! In your face, Reviewer-1, in your face!! Because have you ever heard of the outrageous possibility that someone has coded the program correctly in the first attempt? No, it is ridiculous to imagine so. We encourage you to pause reading at this point and spend some time laughing at the very idea. Hence, by some stroke of ill-luck, the program would either not compile, or, even if it did, there would be some bug in it, which disallows a worst case of $\mathcal{O}(n)$.

Reviewer-2: But, we assume that the program is written correctly, and given the correct implementation we then try to analyze the worst case time complexity.

Aha! Do you, the astute Reader, figure out the glaring flaw in Reviewer-1 and Reviewer-2's arguments? It is paradoxical in a sense, that to find the Worst Case Time Complexity, one has to make a lot of simplifying, optimistic, *impractical* assumptions, which by definition, defeats the definition of "Worst Case". So, the classical definition of Worst Case is, in the worst case, the worst case of the definition accepted. It needs a radical overhaul.

Thus, we present here, for the first time in literature and history, thereby establishing our firm footing as the greatest of all Theoretical Computer Science researchers, a new Definition for measuring Time Complexity of programs: the *Nut-Case Time Complexity*.

NUT-CASE TIME COMPLEXITY: A program has Nut-Case Time Complexity, $\mathcal{N}(f(n))$ if one can imagine any possible scenario, in any possible universe, under any possible assumption, whereby the program can run in time $\Theta(f(n))$.

For the above example of linear search, the Nut-Case Time Complexity is $\mathcal{N}(\infty)$. This is because one needs to repeatedly scan the array *ad infinitum*, because one can never know if any of the elements can change to the element we are searching for. Actually, to be entirely honest, the Nut-Case Time Complexity for Linear Search is $\mathcal{N}(\perp)$, to indicate that it is undefined. This is because if one were to imagine the worst possible case, one can easily envisage a scenario where the computer could completely breakdown, or a Tsunami could cataclysmically expunge the world, or any other morbid scenario that the reader is willing to imagine, which ultimately destroys the array, the program and everything else under consideration, leaving the solution undefined. Clearly, the Nut-Case Time Complexity is far more ~~worse~~ *useful* than merely the Worst Case Time Complexity, and we hereby propose that this become the *de facto* standard of measuring time complexity of programs going forward. Anybody who refuses to see the eye-blistering brilliance of this is clearly a *Nutcase*.

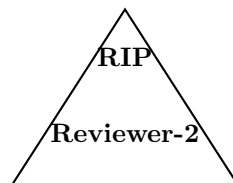
Reviewer-2: You must be a Nutcase to imagine that these kinds of scenarios would materialize to affect the Worst Case Time Complexity of a program.

Author: Thank you for validating the name for this Time Complexity.

Reviewer-2: But you can't— AAAAAAAHHHHH!!!!



Let us observe two minutes of mournful silence to deeply commiserate the loss of Reviewer-2 to a cheetah that suddenly dropped from the sky to devour the Reviewer. Let this serve as a grim reminder of the power of the Nut-Case Time Complexity. A new Reviewer shall *hopefully* be found.



<End Digression>

Now, given the clearly pessimistic nature of Worst Case (and Nut-Case) analyses, we hereby propose, for the greater good of the computing world, a novel architecture, called the Optimistic Processing Engine (OPE), thereby establishing our firm second footing, in addition to being the greatest Theoretical Computer Science researchers, also the greatest Computer Architecture and System researchers. The architectural details of such a machine is left as future work for subsequent SIGBOVIK papers, but a brief idea of the working of the OPE is given here to follow the algorithm we propose.

Assume that a given program $\mathcal{P}$ takes set of inputs $\mathcal{I}$, and gives set of outputs $\mathcal{O}$. We can denote this as $\mathcal{P}(\mathcal{I}) = \mathcal{O}$. The OPE runs $\mathcal{P}$, taking inputs as $\langle \mathcal{I}, \mathcal{O} \rangle$. It then mutates the input $\mathcal{I}$ to become $\mathcal{I}'$ so that $\mathcal{P}(\mathcal{I}') = \mathcal{O}$. Such a machine presents a challenge to analyze Time Complexity, because one can almost view the machine as if it was going back in time to change the input, leading to the invention of the first ever *Time Machine*, firmly establishing our *third* footing as the greatest Scientists of all time! The OPE runs on the most famous law of Software Engineering:

Law of Software Engineering: Any hard enough problem, no matter *how* hard, can be solved *easily*, provided one changes the problem.

The algorithm for finding Triangle Count using OPE is straightforward. Given a graph G , guess the triangle count to be any random number x . Run this on an OPE, resulting in a new graph G' , for which the number of triangles is guaranteed to be x .

Proof of correctness: By definition, *duh*!

Time Complexity: Unknown. Left for future work, and by this, we really mean *future* work.

6 Algorithm-4: Hover Over Planet Earth

It is well-acknowledged that given any problem, looking at it from a distance of 20000 feet can help give a much refreshed perspective, often leading to innovative solutions to seemingly insurmountable problems. What if we actually surmount the problem at hand, and in fact do look at it from a distance of 20000 feet? At this distance, the whole graph simply reduces to a point, for the edges being perfect lines blend into one another, and at a distance of 20000 feet, the number of triangles is simply 0. Output this as the answer, leading to a time complexity of $\mathcal{O}(1)$.

Reviewer-1: 

New Reviewer-2: 

Reviewer-3: 

All Reviewers: But it leads to an incorrect answer!

The author has moved on, far far away from planet earth, to a different dimension, above and beyond the petty confines of triangle counting and every other graph problem that has paled into insignificance, to a new land that offers enticing opportunities, vistas unexplored, worlds unimagined, and a universe full of HOPE...

ACKNOWLEDGEMENTS

I would like to thank a lot of people, even if not for contributing directly to this work, for being the beacon of encouragement and support that was responsible for my spark of creativity.

As usual, when it comes to family, my mother, Geetha Chandrashekar, stands first. Forever and always. Acknowledging other family members: (Late) G S Chandrashekar, (Late) Shanthamma, Sagar, Gayathri, Gouthami, Nikhil, Trupti, Yogashri, Siddhi, Premalatha, (Late) Gururaj, Praveen, Raju, Vijaya Shankar and family, Santosh, (Late) Vinod, Vidya, Alok and family, Shalini Raghavendra M and family, and of course, my two dogs.

Teachers have played the most pivotal role in harnessing my creativity and channeling it in the right directions, which otherwise would have been an undirected flare of light lost to the night. So many teachers, right from childhood, whom I would like to thank. I will name some prominent ones, in chronological order: Dinesh Sir, Prema Ma'am, N S Kumar Sir, Jawahar Sir, Viraj Sir, Srinath Sir, Kavi Mahesh Sir, Prof Govindarajan, Prof Rupesh, Prof Srikant, Prof Jayant, Prof Deepak D'Souza, Prof Arindam, and many others.

And of course, a lot of friends throughout, who have been a staunch pillar of support in the darkest of times: Varun Shenoy, Anjana Pai, Varun Rao, Chittaranjan, Vishal, Kavita, Sunil, Kevin, Jeevan, Pradyumna, Gaurav, Nikhil, Srishty, Hemanta, Shruti Dixit, Kshama, Disha, Harshell, Preeti, Sreenidhi, Nischita, Nikitha, Ritika, Ajay Ashok, Aditya Kamath, Pratheek B, Shweta Pandey, Shilpa, Stanly, Shubham, Akash, Ram and many more. Thank you all for everything, even if thanking you means embarrassing you by naming you in a paper like this!

References

- [1] Pratchett, Terry. *Going Postal*. Vol. 33. Random House, 2011.
- [2] Sorry, the paper doesn't yet exist. It will, someday in the future, HOPEfully.